

1c_bonus-C.md

Below is the Route53 add-on for Bonus-B (Hosted Zone + ACM DNS validation records + app.chewbacca-growl.com ALIAS → ALB). It's written as a Terraform skeleton with Chewbacca comment style.

Add this as `bonus_b_route53.tf`.

Add to `variables.tf` (append)

```
variable "manage_route53_in_terraform" {
  description = "If true, create/manage Route53 hosted zone +
  records in Terraform."
  type        = bool
  default     = true
}
```

```
variable "route53_hosted_zone_id" {
  description = "If manage_route53_in_terraform=false, provide
  existing Hosted Zone ID for domain."
  type        = string
  default     = ""
}
```

Add file: `bonus_b_route53.tf`

Important note about the HTTPS listener

In your earlier `bonus_b.tf`, your HTTPS listener referenced:

```
certificate_arn = aws_acm_certificate_validation.chewbacca_acm_validation01.certificate_arn
```

Now you have two possible validation resources (email/manual vs DNS). For the skeleton, do this pattern:

Keep your original `aws_acm_certificate_validation` (email/manual) if you want

OR switch the listener certificate ARN to the certificate itself and rely on validation dependency

Best skeleton approach (simple + works):

Update HTTPS listener to use:

```
certificate_arn = aws_acm_certificate.chewbacca_acm_cert01.arn
```

...and keep `depends_on` pointing at the DNS validation resource when DNS mode is used.

I'll give you a clean patch below.

Explanation: HTTPS listener is the real hangar bay – TLS terminates here, then traffic goes to private targets.

```
resource "aws_lb_listener" "chewbacca_https_listener01" {
  load_balancer_arn = aws_lb.chewbacca_alb01.arn
  port              = 443
  protocol          = "HTTPS"
  ssl_policy        = "ELBSecurityPolicy-TLS13-1-2-2021-06"
  certificate_arn   = aws_acm_certificate.chewbacca_acm_cert01.arn
}
```

```
  default_action {
    type              = "forward"
    target_group_arn = aws_lb_target_group.chewbacca_tg01.arn
  }
```

TODO: If DNS validation is enabled, ensure validation completes before listener creation.

```
  depends_on = [
    aws_acm_certificate_validation.chewbacca_acm_validation01_dns_bonus
  ]
}
```

If you choose EMAIL validation instead,
you can comment out the depends_on or set certificate_validation_method="EMAIL"
and keep the listener creation after manual validation. (This is a skeleton; you'll get some "learning friction" here.)

Continue bonus_b_route53.tf – ALIAS record app → ALB

```
#####  
# ALIAS record: app.chewbacca-growl.com -> ALB  
#####  
  
# Explanation: This is the holographic sign outside the cantina—app.chewbacca-growl.com points to your ALB.  
resource "aws_route53_record" "chewbacca_app_alias01" {  
  zone_id = local.chewbacca_zone_id  
  name     = local.chewbacca_app_fqdn  
  type     = "A"  
  
  alias {  
    name              = aws_lb.chewbacca_alb01.dns_name  
    zone_id           = aws_lb.chewbacca_alb01.zone_id  
    evaluate_target_health = true  
  }  
}
```

Add outputs (append to outputs.tf)

Explanation: Outputs are the nav computer readout—Chewbacca needs coordinates that humans can paste into browsers.

```
output "chewbacca_route53_zone_id" {  
  value = local.chewbacca_zone_id  
}  
  
output "chewbacca_app_url_https" {  
  value = "https://${var.app_subdomain}.${var.domain_name}"  
}
```

```
}
```

Student verification (CLI)

1) Confirm hosted zone exists (if managed)

```
aws route53 list-hosted-zones-by-name \  
  --dns-name chewbacca-growl.com \  
  --query "HostedZones[].Id"
```

2) Confirm app record exists

```
aws route53 list-resource-record-sets \  
  --hosted-zone-id <ZONE_ID> \  
  --query "ResourceRecordSets[?Name=='app.chewbacca-growl.co  
m.'].]"
```

3) Confirm certificate issued

```
aws acm describe-certificate \  
  --certificate-arn <CERT_ARN> \  
  --query "Certificate.Status"
```

Expected: ISSUED

4) Confirm HTTPS works

```
curl -I https://app.chewbacca-growl.com
```

Expected: HTTP/1.1 200 (or 301 then 200 depending on your ap
p)

What YOU must understand (career point)

This is exactly how companies do it:

DNS points to ingress

TLS via ACM

ALB handles secure public entry

private compute does the work

WAF + alarms defend and alert

When students can Terraform this, they're doing real cloud en

```
gineering.
```

Based on your current files and the Bonus C requirements, here's your step-by-step execution plan.

Lab 1C Bonus C: Route53 DNS + ACM Validation

Overview

This bonus adds Route53 DNS management so your domain (williebright.com) properly resolves to your ALB with validated TLS.

Step 1: Add Variables to [03-variables.tf](#)

Append these to the bottom of your existing variables file:

```
# =====
# BONUS C: Route53 Variables
# =====

variable "manage_route53_in_terraform" {
  description = "If true, create/manage Route53 hosted zone +
records in Terraform."
  type        = bool
```

```

    default      = true
  }

  variable "route53_hosted_zone_id" {
    description = "If manage_route53_in_terraform=false, provide existing Hosted Zone ID for domain."
    type        = string
    default     = ""
  }

```

Step 2: Create New File 04-1cc-route53.tf

Create this file for Route53 resources:

```

#####
# BONUS C: Route53 Hosted Zone + DNS Validation + ALIAS
#####

locals {
  helga_zone_id = var.manage_route53_in_terraform ? aws_route53_zone.helga_zone01[0].zone_id : var.route53_hosted_zone_id
  helga_app_fqdn = "${var.app_subdomain}.${var.domain_name}"
}

#####
# Hosted Zone (if managed by Terraform)
#####

resource "aws_route53_zone" "helga_zone01" {
  count = var.manage_route53_in_terraform ? 1 : 0
  name  = var.domain_name

  tags = { Name = "${var.project_name}-zone01" }
}

```

```
#####
# ACM DNS Validation Records
#####

resource "aws_route53_record" "helga_acm_validation_records01" {
  for_each = var.manage_route53_in_terraform ? {
    for dvo in aws_acm_certificate.helga_acm_cert01.domain_validation_options : dvo.domain_name => {
      name      = dvo.resource_record_name
      type      = dvo.resource_record_type
      record    = dvo.resource_record_value
    }
  } : {}

  zone_id = local.helga_zone_id
  name     = each.value.name
  type     = each.value.type
  ttl      = 300
  records  = [each.value.record]
}

resource "aws_acm_certificate_validation" "helga_acm_validation01_dns" {
  count = var.manage_route53_in_terraform ? 1 : 0

  certificate_arn = aws_acm_certificate.helga_acm_cert01.arn
  validation_record_fqdns = [for record in aws_route53_record.helga_acm_validation_records01 : record.fqdn]
}

#####
# ALIAS Record: app.williebright.com → ALB
#####
```

```

resource "aws_route53_record" "helga_app_alias01" {
  zone_id = local.helga_zone_id
  name     = local.helga_app_fqdn
  type     = "A"

  alias {
    name                = aws_lb.helga_alb01.dns_name
    zone_id             = aws_lb.helga_alb01.zone_id
    evaluate_target_health = true
  }
}

```

Step 3: Update HTTPS Listener in 04-1cb-Main.tf

Find your `aws_lb_listener.helga_https_listener01` resource and update it to depend on DNS validation:

```

resource "aws_lb_listener" "helga_https_listener01" {
  load_balancer_arn = aws_lb.helga_alb01.arn
  port              = 443
  protocol          = "HTTPS"
  ssl_policy        = "ELBSecurityPolicy-TLS13-1-2-2021-06"
  certificate_arn    = aws_acm_certificate.helga_acm_cert01.arn

  default_action {
    type                = "forward"
    target_group_arn = aws_lb_target_group.helga_tg01.arn
  }

  # Ensure DNS validation completes before listener creation
  depends_on = [
    aws_acm_certificate_validation.helga_acm_validation01_dns
  ]
}

```



```
]
}
```

Step 4: Add Outputs to `05-outputs.tf`

Append these to your outputs file:

```
# =====
# BONUS C: Route53 Outputs
# =====

output "helga_route53_zone_id" {
  value = local.helga_zone_id
}

output "helga_app_url_https" {
  value = "https://${var.app_subdomain}.${var.domain_name}"
}
```

Step 5: Apply

```
terraform init
terraform plan
terraform apply
```

Note: After Terraform creates the hosted zone, you must update your domain registrar (where you bought williebright.com) to point to the Route53 Name Servers. You can find them in the AWS Console under Route53 → Hosted Zones → your zone → NS record.

Step 6: CLI Verification (Exact from MD)

1. Confirm Hosted Zone Exists

```
aws route53 list-hosted-zones-by-name \
  --dns-name williebright.com \
  --query "HostedZones[].Id"
```

2. Confirm App Record Exists

```
aws route53 list-resource-record-sets \
  --hosted-zone-id <ZONE_ID_FROM_STEP_1> \
  --query "ResourceRecordSets[?Name=='app.williebright.com.'].]"
```

```
aws route53 list-resource-record-sets \
  --hosted-zone-id /hostedzone/<hosted Zone ID>\
  --query "ResourceRecordSets[?Name=='app.williebright.com.'].]"
```

3. Confirm Certificate Issued

```
aws acm describe-certificate \
  --certificate-arn <YOUR_CERT_ARN> \
  --query "Certificate.Status"
```

```
aws acm describe-certificate \
  --certificate-arn arn:aws:acm:us-east-2:<account number here>:certificate/
  --query "Certificate.Status"
```

Expected: `"ISSUED"`

4. Confirm HTTPS Works

```
curl -I https://app.williebright.com
```

Expected: `HTTP/1.1 200` (or `301` then `200` depending on your app)

Let me know when you're ready to start, and we can walk through each step together.

```
aws route53 list-resource-record-sets \  
--hosted-zone-id <ZONE_ID> \  
--query "ResourceRecordSets[?Name=='app.williebright.com.'].  
{Name:Name,Type:Type}"  
  
/hostedzone/Z076499737UY78K9J1ZVE  
  
aws route53 list-resource-record-sets \  
--hosted-zone-id /hostedzone/Z076499737UY78K9J1ZVE \  
--query "ResourceRecordSets[?Name=='app.williebright.com.'].  
{Name:Name,Type:Type}"
```